Imports & Configuration

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

```
from pyspark.sql.types import *
import pyspark.sql.functions as F
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.master("local[*]").getOrCreate()
```

```
spark.conf.set("spark.sql.shuffle.partitions", "3")
spark.conf.get("spark.sql.shuffle.partitions")
spark.conf.set("spark.sql.adaptive.enabled", "false")
```

```
'3'
```

> ### Definition: salting
>
> - Skew = one key holds most of the rows, so `hash(key) % n` sends them all to **one** partition.
> - **Salt** = a random number 0..N-1 glued onto the key.
> - The join/groupBy key becomes `(value, salt)`, so one hot key becomes N keys that hash to N different partitions.
> - Config here: `shuffle.partitions = 3`, `AQE = false` — AQE off on purpose, or it would split the skewed partition for us and hide the problem.
>
> Salt widens the **key**, not the data. Same rows, more distinct keys to hash.

Simulating Skewed Join

```
df_uniform = spark.createDataFrame([i for i in range(1000000)], IntegerType())
df_uniform.show(5, False)
```

```
+-----+
|value|
+-----+
|0    |
|1    |
|2    |
|3    |
|4    |
+-----+
only showing top 5 rows
```

```
(
    df_uniform
    .withColumn("partition", F.spark_partition_id())
    .groupBy("partition")
    .count()
    .orderBy("partition")
    .show(15, False)
)
```

```
+---------+-----+
|partition|count|
+---------+-----+
|0        |82944|
|1        |82944|
|2        |83968|
|3        |82944|
|4        |83968|
|5        |82944|
|6        |82944|
|7        |83968|
|8        |82944|
|9        |83968|
|10       |82944|
|11       |83520|
+---------+-----+
```

```
df0 = spark.createDataFrame([0] * 999990, IntegerType()).repartition(1)
df1 = spark.createDataFrame([1] * 15, IntegerType()).repartition(1)
df2 = spark.createDataFrame([2] * 10, IntegerType()).repartition(1)
df3 = spark.createDataFrame([3] * 5, IntegerType()).repartition(1)
df_skew = df0.union(df1).union(df2).union(df3)
df_skew.show(5, False)
```

```
+-----+
|value|
+-----+
|0    |
|0    |
|0    |
|0    |
|0    |
+-----+
only showing top 5 rows
```

```
(
    df_skew
    .withColumn("partition", F.spark_partition_id())
    .groupBy("partition")
    .count()
    .orderBy("partition")
    .show()
)
```

```
+---------+-------+
|partition|  count|
+---------+-------+
|        0|1000020| ①|
+---------+-------+
```

✍ What the skewed input looks like

match the circled numbers above

- ① **partition 0** ⟶ all 1,000,020 rows — one partition, nothing else

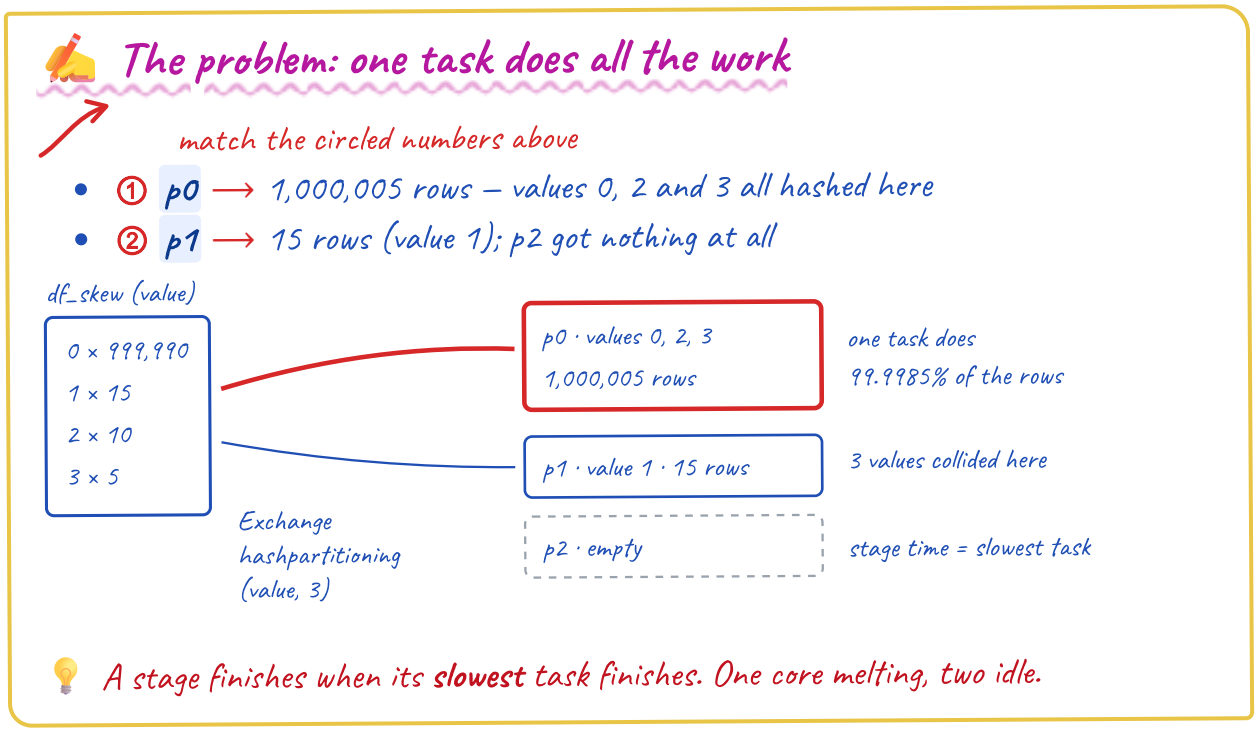
Each piece was built with **.repartition(1)** and then **union**ed, so the whole frame starts life in a single partition. Remember this — it changes what the aggregation demo can prove.

💡 This is **input** layout. The shuffle in the join below is what turns it into **task** skew.

```
df_joined_c1 = df_skew.join(df_uniform, "value", 'inner')
```

```
(
    df_joined_c1
    .withColumn("partition", F.spark_partition_id())
    .groupBy("partition")
    .count()
    .show(5, False)
)
```

```
+---------+-------+
|partition|count  |
+---------+-------+
|0        |1000005|
|1        |15     |
+---------+-------+
```



Simulating Uniform Distribution Through Salting

```
SALT_NUMBER = int(spark.conf.get("spark.sql.shuffle.partitions"))
SALT_NUMBER
```

```
3
```

```
df_skew = df_skew.withColumn("salt", (F.rand() * SALT_NUMBER).cast("int"))
```

```
df_skew.show(10, truncate=False)
```

```
+-----+----+
|value|salt| ①|
+-----+----+
|0    |0   |
|0    |0   |
|0    |0   |
|0    |2   |
|0    |0   |
|0    |0   |
|0    |0   |
|0    |0   |
|0    |1   |
|0    |1   |
+-----+----+
only showing top 10 rows
```

Step 1 · salt the skewed side

match the circled numbers above

- ① `(F.rand() * SALT_NUMBER).cast("int")` ⟶ random 0, 1 or 2 per row
- ② value 0 ⟶ 0/0, 0/1, 0/2 ⟶ one hot key becomes 3 keys

rand() is random, not round-robin — shares come out near-equal, never exact.

```
df_uniform = (
    df_uniform
    .withColumn("salt_values", F.array([F.lit(i) for i in range(SALT_NUMBER)]))
    .withColumn("salt", F.explode(F.col("salt_values")))
)
```

```
df_uniform.show(10, truncate=False)
```

```
+-----+-----------+----+
|value|salt_values|salt|
+-----+-----------+----+
|0    |[0, 1, 2]  |0   |
|0    |[0, 1, 2]  |1   |
|0    |[0, 1, 2]  |2   |
|1    |[0, 1, 2]  |0   |
|1    |[0, 1, 2]  |1   |
|1    |[0, 1, 2]  |2   |
|2    |[0, 1, 2]  |0   |
|2    |[0, 1, 2]  |1   |
|2    |[0, 1, 2]  |2   |
|3    |[0, 1, 2] ①|0   |
+-----+-----------+----+
only showing top 10 rows
```

✍️ Step 2 · explode the small side

match the circled numbers above

- ① `F.array([lit(0), lit(1), lit(2)])` ⟶ give every row the full salt list
- ② `F.explode(salt_values)` ⟶ 1 row ⟶ 3 rows, one per salt

Why: the skewed side picked its salt at **random**. The small side cannot guess which one, so it must carry **all** of them or the join loses rows.

⚠ **Gotcha:** skip the explode and the join silently returns fewer rows — no error, just wrong answers.

💡 Explode the **smaller** side: it is multiplied by SALT_NUMBER.

```
df_joined = df_skew.join(df_uniform, ["value", "salt"], 'inner')
```

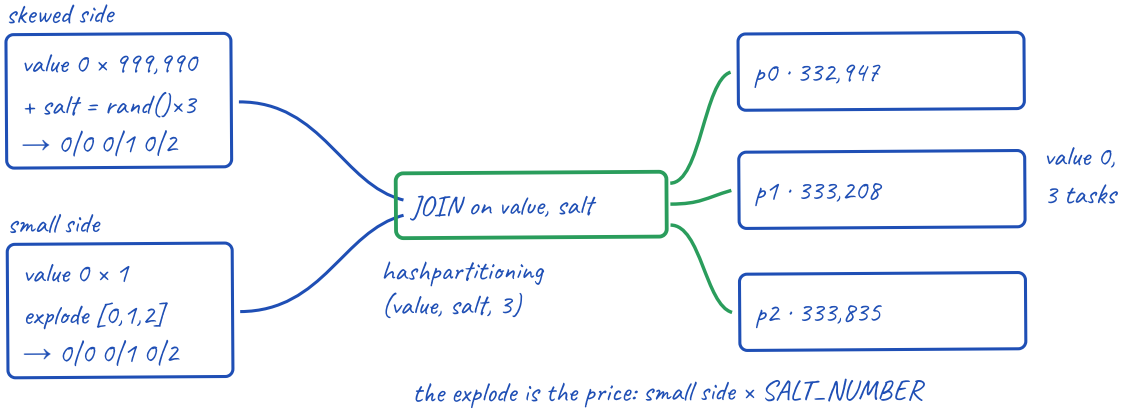
```
(
    df_joined
    .withColumn("partition", F.spark_partition_id())
    .groupBy("value", "partition")
    .count()
    .orderBy("value", "partition")
    .show()
)
```

```
+-----+---------+------+
|value|partition| count|
+-----+---------+------+
|    0|        0|332947| ①|
|    0|        1|333208|
|    0|        2|333835|
|    1|        0|     7|
|    1|        1|     8|
|    2|        0|     5|
|    2|        1|     2|
|    2|        2|     3|
|    3|        0|     3|
|    3|        1|     2|
+-----+---------+------+
```

Step 3 · join on (value, salt)

match the circled numbers above

- ① **value 0 → p0, p1, p2** → 332,947 / 333,208 / 333,835 — near thirds



Heaviest task: 1,000,005 → 333,835 rows. **3× smaller** — exactly SALT_NUMBER.

Salting In Aggregations

```
df_skew.groupBy("value").count().show()
```

```
+-----+------+
|value| count|
+-----+------+
|    0|999990| ①
|    1|    15|
|    2|    10|
|    3|     5|
+-----+------+
```

Same disease in groupBy

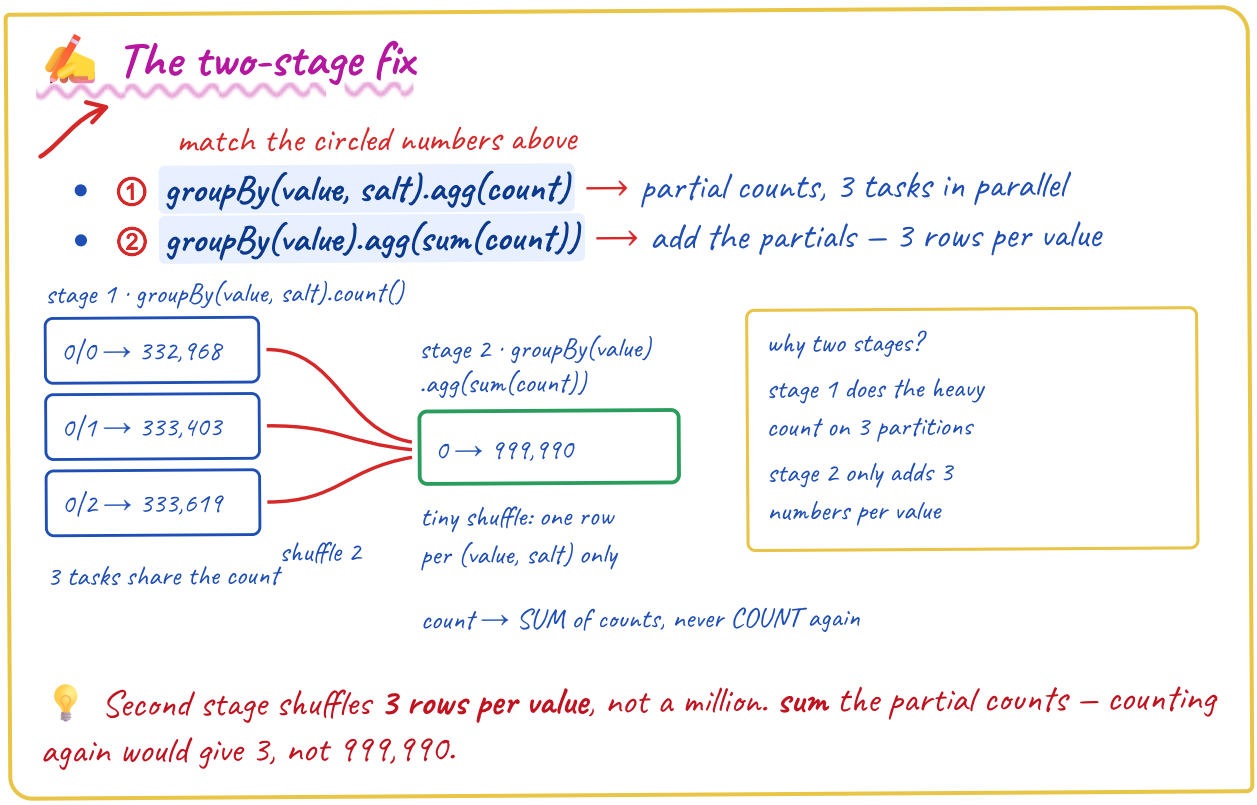
match the circled numbers above

- ① **groupBy("value").count()** → all 999,990 zeros shuffle to one partition

Right answer, wrong shape: one task counts a million rows while the others sit idle.

```
(
    df_skew
    .withColumn("salt", (F.rand() * SALT_NUMBER).cast("int"))
    .groupBy("value", "salt")
    .agg(F.count("value").alias("count"))
    .groupBy("value")
    .agg(F.sum("count").alias("count"))
    .show()
)
```

```
+-----+------+
|value| count|
+-----+------+
|    0|999990| ①|
|    1|    15|
|    2|    10|
|    3|     5|
+-----+------+
```




```
spark.stop()
```

Salting — the parts the notebook does not print

Query plans, the join/aggregation split, and the lines to say out loud. All numbers below come from the run on this machine (Spark 4.3.0, shuffle.partitions = 3).

Salting makes the key wider so one hot key hashes to N partitions.

Join: salt one side, explode the other, join on (value, salt). Aggregation: groupBy(value, salt) → then groupBy(value) and SUM the partials.

1 · Query plans: what actually changed

Skewed join (before)

```
*(8) Project [value]
+- *(8) SortMergeJoin [value], [value], Inner
   :- *(5) Sort [value ASC NULLS FIRST], false, 0
   :  +- Exchange hashpartitioning(value, 3)
   :     +- Union
   +- *(7) Sort [value ASC NULLS FIRST], false, 0
      +- Exchange hashpartitioning(value, 3)
      ... 14 x (Exchange SinglePartition + Scan ExistingRDD)
```

Salted join (after)

```
*(9) Project [value, salt, [0,1,2] AS salt_values]
+- *(9) SortMergeJoin [value, salt], [value, salt], Inner
   :- *(6) Sort [value ASC NULLS FIRST, salt ASC NULLS FIRST],
   :  +- Exchange hashpartitioning(value, salt, 3)
   :     +- *(5) Filter (isnotnull(value) AND isnotnull(salt))
   :        +- *(5) Project [value, cast((rand(-91912021034128
   :           +- Union
   +- *(8) Sort [value ASC NULLS FIRST, salt ASC NULLS FIRST],
      +- Exchange hashpartitioning(value, salt, 3)
         +- *(7) Generate explode([0,1,2]), [value], false, [sa
      ... 10 x (Exchange SinglePartition + Scan ExistingRDD)
```

Both plans shuffle — salting does not remove the `Exchange`. What changes is the **partitioning expression** inside it:

```
before:  Exchange hashpartitioning(value, 3)
after :  Exchange hashpartitioning(value, salt, 3)
```

Same `SortMergeJoin`, same number of exchanges — one extra column in the hash, and the rows for value 0 stop sharing a destination. The salted plan also carries `cast((rand(...) * 3.0) as int) AS salt` on the skewed side and the exploded `salt_values` on the other.

> Read the `Exchange hashpartitioning(...)` line first: the columns listed there are the ones that decide your skew.

2 · ★ Aggregation plans: what this notebook does *not* prove

groupBy(value).count()

```
*(5) HashAggregate(keys=[value], functions=[count(1)])
+- Union
      ... 8 x (Exchange SinglePartition + Scan ExistingRDD)
```

Two-stage salted

```
*(5) HashAggregate(keys=[value], functions=[sum(count)])
+- *(5) HashAggregate(keys=[value, salt], functions=[count(valu
   +- *(5) Project [value, cast((rand(1835784249383247432) * 3
      +- Union
      ... 8 x (Exchange SinglePartition + Scan ExistingRDD)
```

> **Neither aggregation plan contains a `hashpartitioning` exchange.** The only `Exchange` lines are `SinglePartition, REPARTITION_BY_NUM` — those come from the `.repartition(1)` calls in cell 8, not from the groupBy.

Cell 9 shows why: after the union **every one of the 1,000,020 rows sits in partition 0**. When all rows for a key are already together, Spark does not need to shuffle to group them, so it plans a single `HashAggregate`. In the salted version the two `HashAggregate` steps are stacked inside the **same** whole-stage (`*(5)`) with no exchange between them.

So the aggregation half of this notebook demonstrates the **API pattern** (partial counts → sum) and returns the right answer, but on this toy data it is not fixing a shuffle, because there is no shuffle to fix. The join half *does* demonstrate the real thing — see section 1, where the partitioning expression visibly gains the salt column.

> **Gotcha:** the second stage must be `sum(count)`. Using `count()` again counts the partial rows (3 per value) instead of the original rows (999,990).

> To see aggregation salting actually pay off, build the skewed side across many partitions (drop the `repartition(1)`) so the groupBy has to shuffle.

3 · Join salting vs aggregation salting

	Join	Aggregation
Salt the	skewed side, random	skewed side, random
Other side	explode across all salts — **required**	nothing to explode
Key	join on `(value, salt)`	groupBy `(value, salt)`, then `(value)`
Extra cost	small side × N rows	a second, tiny shuffle
Fails silently if	you forget the explode → missing rows	you re-count instead of sum → wrong counts

> Joins pay in **rows**; aggregations pay in **an extra stage**.

4 · Choosing SALT_NUMBER

The notebook sets it to `spark.sql.shuffle.partitions` (= 3), which is a sane default: it is the most spreading you can use before salts start sharing a partition anyway.

Too small	Too large
hot key still lands on a few partitions; skew survives	small side explodes to N× its size; many tiny tasks, scheduler overhead

> Start at the shuffle-partition count, then raise only while the heaviest task keeps shrinking.

5 · Say it out loud (interview)

“What is data skew and how do you fix it?”

> *Skew is when one key holds most of the rows, so `hash(key) % n` routes them to a single partition and one task runs long after the rest finish. I would first try AQE skew join or a broadcast join. If the key is genuinely hot and both sides are large, I salt: add a random 0..N-1 to the skewed side, explode the other side across the same range, and join on key plus salt.*

“What does salting cost?”

> *The small side is duplicated N times, so the shuffle carries more rows. For aggregations it is cheaper — a second shuffle of only N rows per key. You are trading total work for parallelism, which only pays when one task is the bottleneck.*

“Why explode instead of a random salt on both sides?”

> *Two independent random draws rarely match, so most rows would find no partner and the join would quietly drop them. The explode guarantees every salt value exists on the other side.*

6 · Before you salt

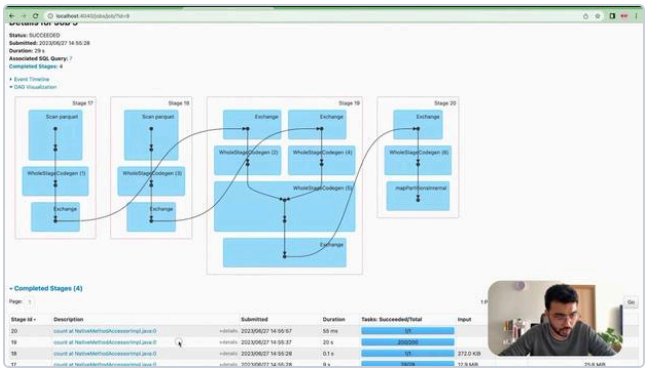
Try first	Why
`spark.sql.adaptive.skewJoin.enabled`	AQE splits an oversized partition for you, no code change. This notebook sets `adaptive.enabled=false` precisely so the skew stays visible.
Broadcast join	no shuffle at all when one side fits in memory
Filter / pre-aggregate	cheapest fix is fewer rows reaching the shuffle

> Salting is manual, changes your code, and adds rows. It is the answer when AQE and broadcast are off the table — not the first reach.

How to read the DAG

Every stage boundary in this picture is a shuffle. Counting them tells you where the time goes before you measure anything.

9 · The picture



Job 9 after it finished: four stages, three Exchange boundaries, and the stage table underneath.

Box	What it is
Outer box (Stage 17…20)	one **stage** — a run of work with no shuffle inside it
`Scan parquet`	reading the source files
`WholeStageCodegen(n)`	operators fused into a single generated Java method — filters, projects, joins, partial aggregates
`Exchange`	the **shuffle**: written at the end of one stage, read at the start of the next
`mapPartitionsInternal`	a last per-partition step, here the final count

Curved line across boxes	a shuffle dependency — the arrow Spark waits on

10 · Reading it in order

Stages 17 and 18 each scan a side and end in `Exchange`: both sides are being repartitioned by the join key. They are independent, so they run at the same time — that is why the running shot says **Active Stages: 2**.

Stage 19 opens with *two* `Exchange` boxes, one per side. This is the post-shuffle side of the join, and it cannot start until both upstream shuffles are fully written — hence **Pending Stages: 2**. **This stage is where a skewed key lands.** It ends in a third `Exchange`, the shuffle for the aggregation.

Stage 20 reads that last exchange and produces the single result row.

> **A stage cannot start until every shuffle it reads is finished writing.** One slow task in stage 17 holds up all 200 tasks of stage 19, even though they are unrelated to each other. That is why a single straggler costs the whole job.

11 · The stage table underneath the DAG

Stage	Tasks	Duration	Input	Reads as
17	28	9 s	12.9 MiB	scan + shuffle write, the big side
18	1	0.1 s	272 KiB	scan + shuffle write, the small side
19	**200**	**20 s**	—	the join itself — two-thirds of a 29 s job
20	1	55 ms	—	final aggregate

200 tasks taking 20 s is not automatically skew — 200 is just `spark.sql.shuffle.partitions`. The question is whether those 200 tasks finished together or one ran for 19 of the 20 seconds. The DAG cannot tell you; the task table can.

> DAG tells you **where** the shuffles are. The task table tells you **whether one of them is skewed**. Always look at both.

Reading it in the Spark UI

Captured live from this run's own Spark UI — app local-1793806777819, local[4], shuffle.partitions = 3. (The UI picks 4041 when 4040 is already taken.)

7 · What the numbers actually said

Join	Stage	Tasks	Shuffle read	Records per task	Task ms	Spill
skewed join	6	3	4.9 MiB	1,332,792 / 334,041 / 0	381 / 206 / 103	none
salted join	14	3	20.5 MiB	1,334,452 / 1,333,334 / 1,332,234	370 / 368 / 346	none

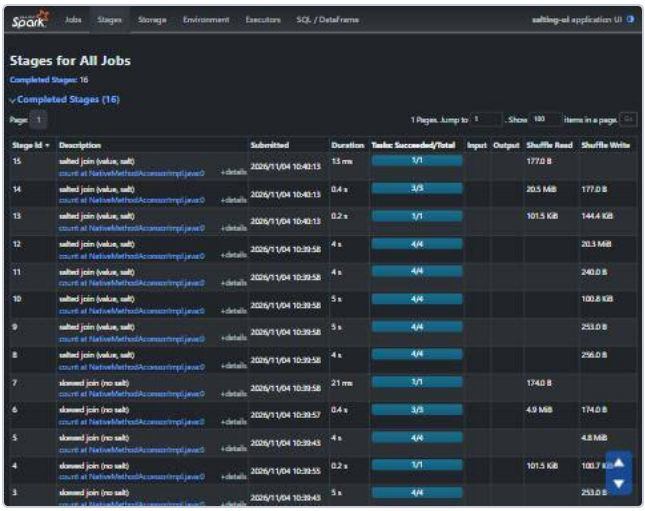
Read the records column, not the clock. Unsalted, the three tasks got **1,332,792 / 334,041 / 0** records — one task did about 80% of the work and one got nothing at all. Salted, the same three tasks got **1,334,452 / 1,333,334 / 1,332,234**: even to within 0.2%.

> **The wall clock barely moved: 16.3 s vs 15.3 s, and nothing spilled.** At 1M small rows on local[4] a 4× heavier task still finishes in 381 ms, so there is no straggler to feel. The distribution is genuinely fixed; the machine was never under enough pressure for it to pay. Skew costs real time when a partition stops fitting in a task's memory budget — that is when the straggler and the spill appear together.

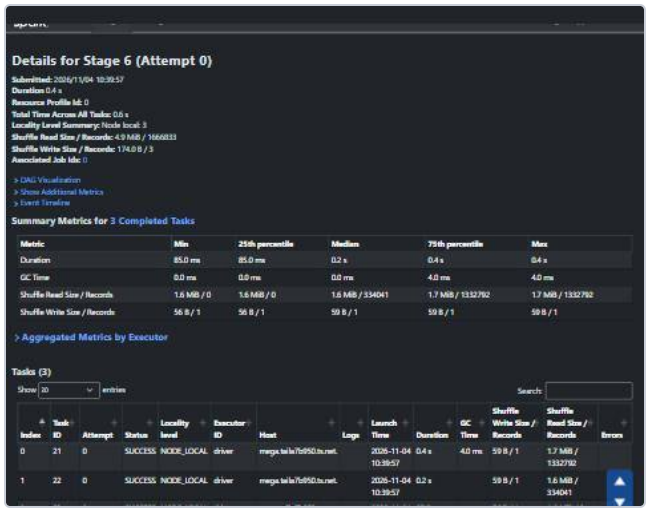
The cost of salting *is* visible though: shuffle read goes from **4.9 MiB** to **20.5 MiB**, roughly 4×, because the small side was exploded across every salt. You pay that on every run; you only collect on it when a task is the bottleneck.

> Spread is proved by **records per task**; the payoff is proved by **duration**. On a laptop you will often get the first without the second.

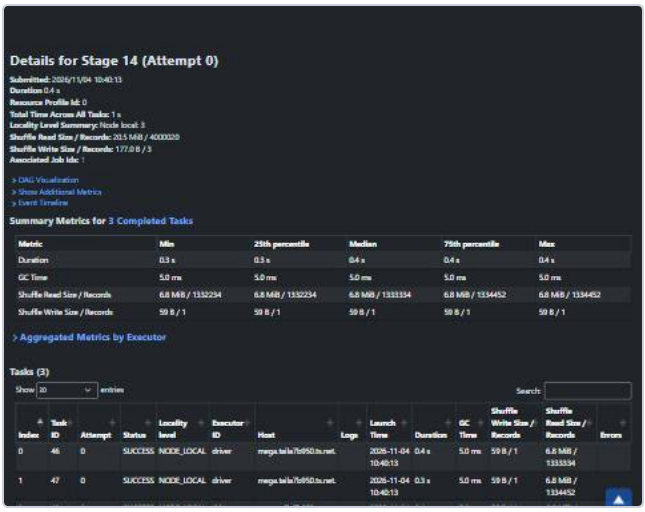
8 · Where to click



Stages tab. Both joins ran in one app, tagged by job group, so the rows line up.



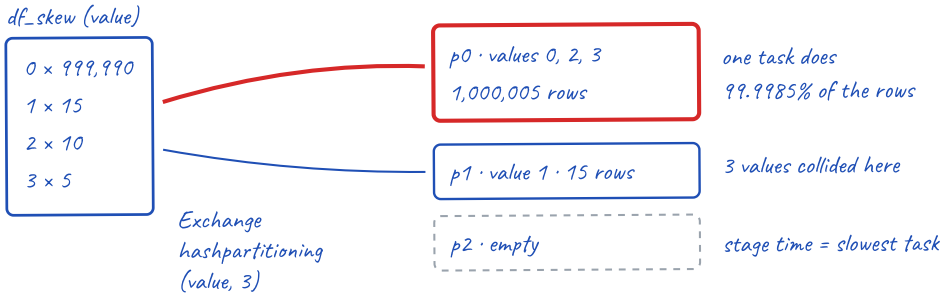
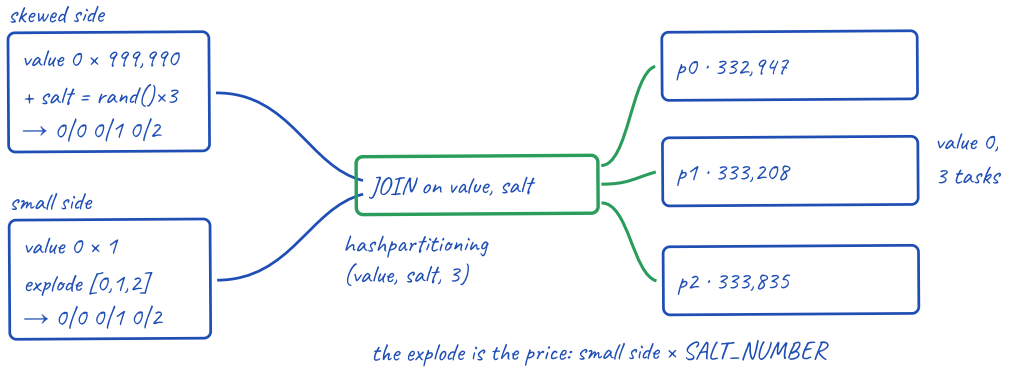
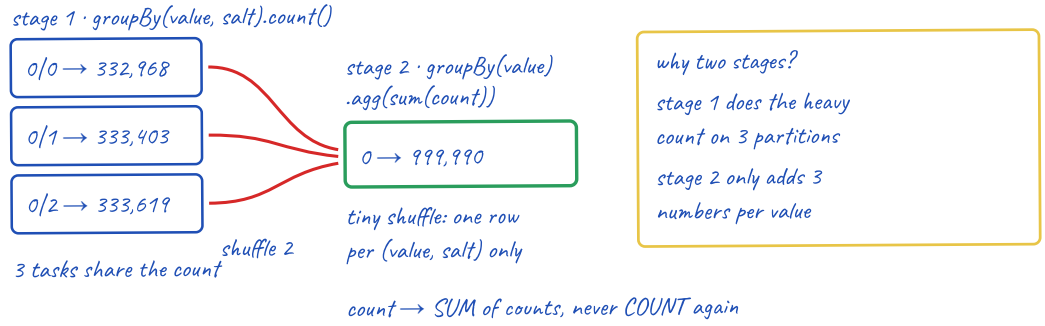
Skewed join, stage 6. Summary metrics: Shuffle Read records go Min **0** → Max **1,332,792**.



Salted join, stage 14. Same row reads Min **1,332,234** → Max **1,334,452**.

> Jobs → the slow one → its longest stage → the task table, sorted by Duration. Four clicks to the straggler.

Salting — one page

Technique	Rule	This notebook	Shape
Detect skew `spark_partition_id()`	groupBy(partition) .count() — one row >> all the rest	after the join: p0 = 1,000,005 p1 = 15, p2 = 0	
Salt a join `rand()*N` + `explode`	hash key becomes (value, salt); small side × N	heaviest task 1,000,005 → 333,835	
Salt an aggregation `groupBy(v,s)` then `groupBy(v)`	partial counts, then **sum** — never count twice	3 partials → 999,990 (no shuffle here: see page 2 note)	

💡 Same question every time: **what is in the hashpartitioning(...) line, and is one value most of the rows?**